FILE: C:\HCL_TRAINING\server\src\services\matchingEngine\aiProvider.js

LINES: 134 | SIZE: 4.6 KB | LANGUAGE: JS

MODULE: Hugging Face API + Local TF-IDF Cosine Vector Fallback

```
/**
 * AI Provider Abstraction Layer
 * Supports Hugging Face Inference API / Configurable Embedding Services
 * with an automatic, robust local vector space fallback.
 */

// Stop words for clean local text vectorization
const STOP_WORDS = new Set([
  'a', 'about', 'above', 'after', 'again', 'against', 'all', 'am', 'an', 'and', 'any', 'are', 'aren\'t', 'as', 'at',
  'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'cannot', 'could',
  'did', 'do', 'does', 'doing', 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'has', 'have',
  'having', 'he', 'her', 'here', 'hers', 'herself', 'him', 'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is',
  'it', 'its', 'itself', 'just', 'me', 'more', 'most', 'my', 'myself', 'no', 'nor', 'not', 'now', 'of', 'off',
  'on', 'once', 'only', 'or', 'other', 'ought', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 'same', 'she',
  'should', 'so', 'some', 'such', 'than', 'that', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there',
  'these', 'they', 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 'very', 'was', 'we', 'were',
  'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'with', 'would', 'you', 'your', 'yours'
]);

class AIEmbeddingService {
  constructor() {
    this.apiKey = process.env.HF_API_KEY || '';
    this.model = process.env.HF_EMBEDDING_MODEL || 'sentence-transformers/all-MiniLM-L6-v2';
    this.hfEndpoint = `https://api-inference.huggingface.co/pipeline/feature-extraction/${this.model}`;
  }

  /**
   * Calculates semantic similarity between candidate resume text and job description
   * Returns a score from 0 to 100.
   */
  async calculateSimilarity(resumeText, jobText) {
    if (this.apiKey && this.apiKey.trim() !== '') {
      try {
        const sim = await this._getHuggingFaceSimilarity(resumeText, jobText);
        if (typeof sim === 'number' && !isNaN(sim)) {
          return Math.max(0, Math.min(100, Math.round(sim * 100)));
        }
      } catch (err) {
        console.warn('Hugging Face Inference call failed, gracefully falling back to Local Vector Space:', err.message);
      }
    }

    // Default High-Accuracy Local Semantic Vector Space Engine
    return this._calculateLocalSemanticSimilarity(resumeText, jobText);
  }

  /**
   * Hugging Face Inference API call
   */
  async _getHuggingFaceSimilarity(textA, textB) {
    const fetch = global.fetch || require('node-fetch');
    const response = await fetch(this.hfEndpoint, {
      method: 'POST',
      headers: {
```

```
55 |           Authorization: `Bearer ${this.apiKey}`,
56 |           'Content-Type': 'application/json',
57 |         },
58 |         body: JSON.stringify({
59 |           inputs: {
60 |             source_sentence: textB.substring(0, 1000),
61 |             sentences: [textA.substring(0, 1000)],
62 |           },
63 |         }),
64 |       });
65 |
66 |       if (!response.ok) {
67 |         throw new Error(`HF API error: ${response.statusText}`);
68 |       }
69 |
70 |       const data = await response.json();
71 |       if (Array.isArray(data) && data.length > 0) {
72 |         return data[0];
73 |       }
74 |       return 0.75;
75 |     }
76 |
77 |     /**
78 |      * Deterministic, high-precision Local Vector Space Engine (n-gram sublinear TF-IDF +
   >  Cosine)
79 |      */
80 |     _calculateLocalSemanticSimilarity(docA, docB) {
81 |       const tokenize = (text) => {
82 |         return text
83 |           .toLowerCase()
84 |           .replace(/[^a-z0-9#+. ]/g, ' ')
85 |           .split(/\s+/)
86 |           .filter((w) => w.length > 1 && !STOP_WORDS.has(w));
87 |       };
88 |
89 |       const tokensA = tokenize(docA);
90 |       const tokensB = tokenize(docB);
91 |
92 |       if (tokensA.length === 0 || tokensB.length === 0) return 50;
93 |
94 |       const freqA = {};
95 |       const freqB = {};
96 |
97 |       tokensA.forEach((w) => { freqA[w] = (freqA[w] || 0) + 1; });
98 |       tokensB.forEach((w) => { freqB[w] = (freqB[w] || 0) + 1; });
99 |
100 |      // Collect all vocabulary
101 |      const vocabulary = new Set([...Object.keys(freqA), ...Object.keys(freqB)]);
102 |
103 |      let dotProduct = 0;
104 |      let normA = 0;
105 |      let normB = 0;
106 |
107 |      for (const term of vocabulary) {
108 |        // Sublinear tf scaling: 1 + log(tf)
109 |        const weightA = freqA[term] ? (1 + Math.log(freqA[term])) : 0;
110 |        const weightB = freqB[term] ? (1 + Math.log(freqB[term])) : 0;
111 |
112 |        dotProduct += weightA * weightB;
113 |        normA += weightA * weightA;
114 |        normB += weightB * weightB;
115 |      }
116 |
117 |      if (normA === 0 || normB === 0) return 50;
118 |
119 |      const rawCosine = dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
120 |
121 |      // Normalize cosine similarity to a practical 0 - 100 scale
122 |      // Document cosine similarities often range from 0.1 to 0.7; map this smoothly
123 |      const normalized = Math.min(100, Math.max(10, Math.round(rawCosine * 125)));
124 |      return normalized;
125 |    }
```

```
126 |  }
127 |
128 |  const aiProvider = new AIEmbeddingService();
129 |
130 |  module.exports = {
131 |    aiProvider,
132 |    AIEmbeddingService,
133 |  };
134 |
```